
CS 301

High-Performance Computing

Lab 03 - Interpolation of Scattered Data on a Structured Grid

Prem Kukadiya (202301452)
Suryadeepsinh Gohil (202301463)

February 24, 2026

Contents

1	Introduction	3
2	Context	3
3	Hardware Specifications	4
3.1	Lab PC Hardware Specifications	4
3.2	HPC Cluster Hardware Specifications	4
4	Methodology	4
5	Results: Performance Plots	5
6	Answers to Lab Questions and Analysis	7
7	Conclusion	10

1 Introduction

In this lab, we investigate the performance characteristics of mapping randomly scattered 2D spatial points onto a structured computational mesh using Bilinear Interpolation. For every scattered particle within the domain, the algorithm identifies the four nearest bounding grid nodes and distributes a localized value ($f_i = 1$) to them based on fractional, area-weighted distances.

We analyze how memory access patterns and hardware-level behaviors dictate overall execution speed. Specifically, we benchmark:

- The performance penalty of complex data structures (like linked lists or bins) versus the spatial locality benefits of a simple, contiguous flat array.
- The utilization of the CPU's Hardware Prefetcher when traversing massive particle datasets.
- The computational impact of floating-point arithmetic optimization (replacing division with pre-computed inverse multiplication) and branchless programming (using ternary operators instead of standard `if` statements to bypass branch prediction penalties).

By measuring the processing time for datasets scaling up to 20 million points across multiple iterations, we determine the most cache-efficient implementation. Benchmarks are recorded on both a consumer-grade Lab PC and an HPC Cluster to observe hardware-specific scaling.

2 Context

This assignment exposes the severe memory-bound bottlenecks inherent in processing massive, unorganized datasets. The fundamental parameters of our implementation are:

- **Brief description of the problem:** A continuous domain of 1×1 units contains up to $N = 20,000,000$ scattered particles. These must be mapped to an $M \times M$ discrete grid. Each particle updates four separate memory locations on the grid via area-weighted fractions.
- **Complexity of the algorithm (serial):** The time complexity is exactly $O(N \cdot \text{Maxiter})$, as the algorithm executes a strictly bounded set of arithmetic operations for each point. The space complexity is $O(N + M^2)$, required to hold the coordinate array of particles and the output grid.
- **Compute to memory access ratio:** To process a single particle, the CPU performs exactly 22 basic arithmetic operations and 2 logical checks. In exchange, it must read 2 `double` values from main memory (x, y) and execute 4 read-modify-write operations on the grid array. This low arithmetic intensity makes the algorithm heavily dependent on memory bandwidth.
- **Memory-bound vs compute-bound:** This task is heavily **memory-bound**. In the largest configuration, the particle array consumes over 300 MB of RAM, vastly exceeding the L3 cache. If accessed non-sequentially, the CPU stalls while waiting for data fetches.
- **Optimization strategy:** To minimize execution time, our optimized code (`utils.cpp`) utilizes:

1. **Linear Access & Hardware Prefetching:** We store points in a flat structure array and iterate sequentially. The CPU detects this linear pattern and prefetches upcoming particles into the L1 cache before they are explicitly requested.
2. **Algorithmic Offloading:** Repeated calculations (e.g., ' $\mathbf{dx} * \mathbf{dy}$ ', ' $1.0 / \mathbf{NX}$ ') are pulled completely outside the particle loop.
3. **Fast Indexing & Branchless Bounds:** Division operations are replaced by multiplying by the inverse (e.g., ' $\mathbf{x} * \mathbf{inv_dx}$ '). Grid boundary conditions are clamped using one-line ternary operators to prevent CPU branch-prediction stalls.

3 Hardware Specifications

The architecture of the systems was verified using the `lscpu` command.

3.1 Lab PC Hardware Specifications

Table 1: Hardware Specifications for Lab PC (12th Gen Intel i5)

Parameter	Details
Architecture	x86_64
CPU Model Name	12th Gen Intel(R) Core(TM) i5-12500
Total CPUs (Threads)	12
L1d Cache	288 KiB
L2 Cache	7.5 MiB
L3 Cache	18 MiB

3.2 HPC Cluster Hardware Specifications

The benchmarking was also conducted on the HPC Cluster to evaluate performance on a highly parallel architecture:

Table 2: Hardware and Software Specifications (HPC Cluster)

Parameter	Details
CPU Model Name	Intel(R) Xeon(R) CPU E5-2620 v3 @ 2.40GHz
No. of Cores (Threads)	12 Cores / 24 Threads (Dual-socket)
L1d Cache	32 KiB
L2 Cache	256 KiB
L3 Cache	15360 KiB (15 MiB)
Compiler Used	GCC (g++)
Optimization Flags	-O3

4 Methodology

A C++ application was developed to ingest binary files containing the randomly distributed (x_i, y_i) coordinates. The program dynamically allocates a 1D contiguous block of memory to represent

the 2D computational grid ($N_x \times N_y$). The core interpolation function executes over a loop of `Maxiter = 10` to simulate a time-stepping model. Execution time is recorded strictly around the interpolation function using `clock()` high-precision timers.

Input, Output, and Accuracy Details:

- **Input parameters:** The program evaluates five configurations (labeled a through e), testing arrays from 0.9 million up to 20 million points, with underlying meshes ranging from 250×100 to 1000×400 .
- **Output:** The program dumps the final populated mesh to `Mesh.out` and outputs the aggregated interpolation time to the terminal.
- **Accuracy Verification:** Algorithmic correctness was strictly enforced. Prior to benchmarking, the outputs of our optimized `utils.cpp` implementation were compared against the provided reference `test_output.txt` utilizing the Linux `diff -w` utility to ensure zero floating-point discrepancies.

5 Results: Performance Plots

The following plots record the total serial execution time (in seconds) for 10 iterations across the five different problem sizes.

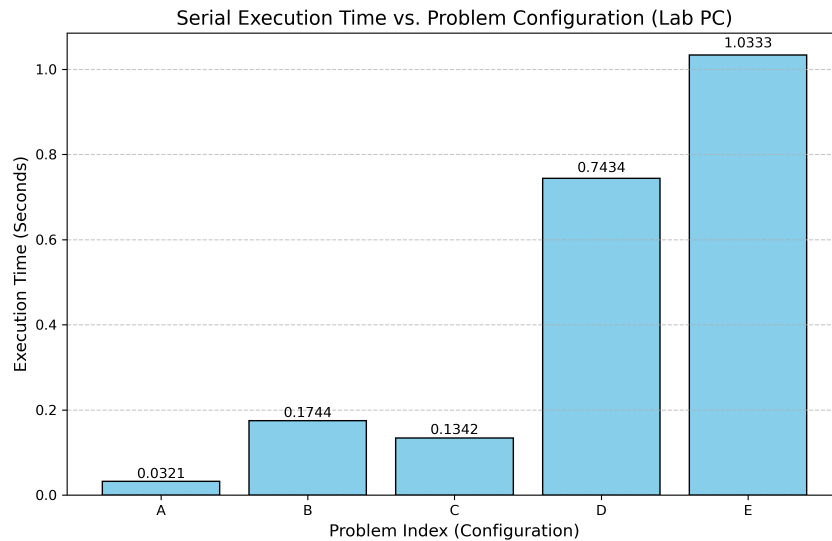


Figure 1: Serial Execution Time vs. Problem Configuration on Lab PC.

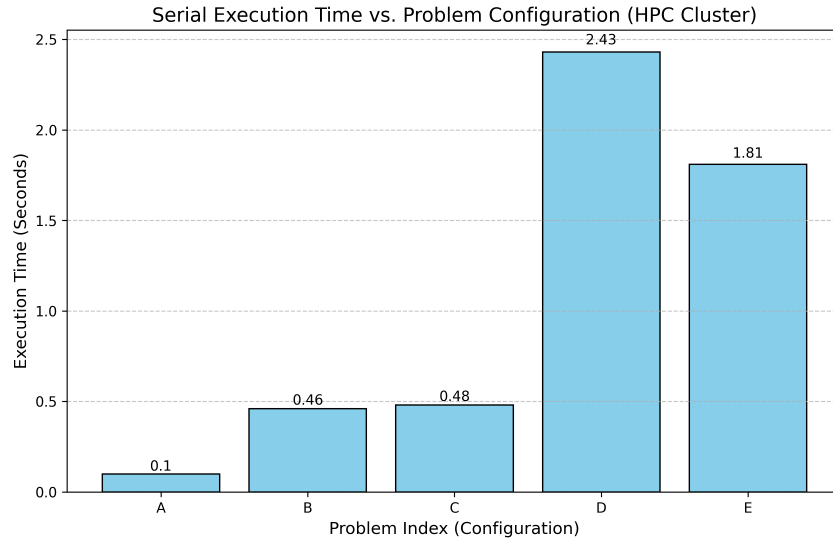


Figure 2: Serial Execution Time vs. Problem Configuration on HPC Cluster.

The grouped bar chart below provides a direct comparison of the total serial execution time. It clearly illustrates the hardware architecture gap between a high-clock-speed desktop environment and a lower-clock-speed, massively multi-core HPC environment when executing single-threaded applications.

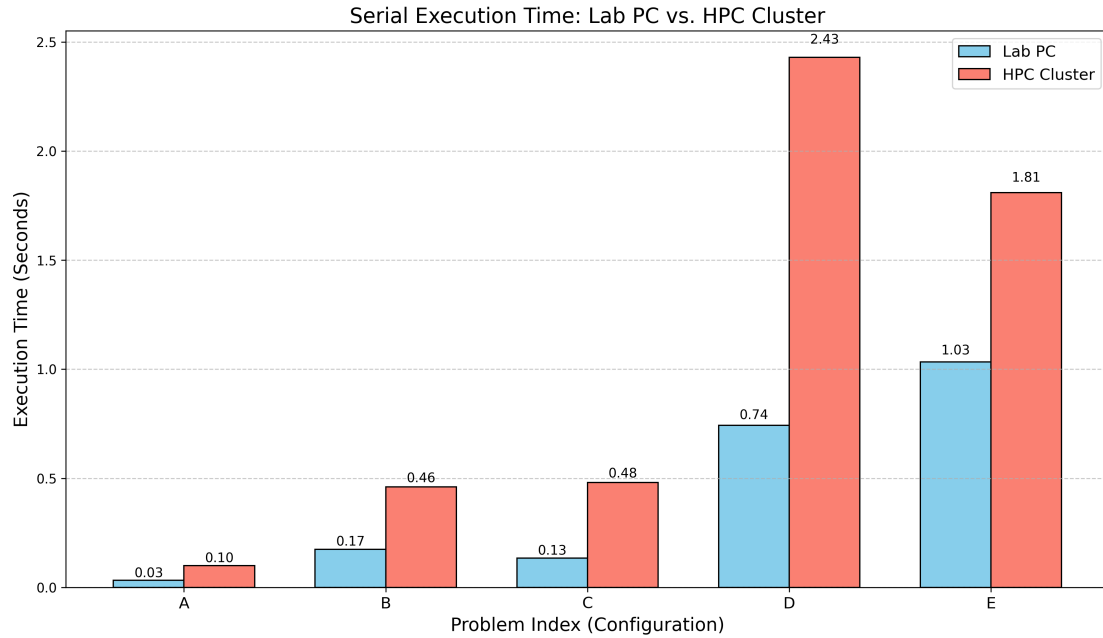


Figure 3: Serial Execution Time Comparison: Lab PC vs. HPC Cluster.

6 Answers to Lab Questions and Analysis

1. Theoretical Time Complexity

Because the algorithm loops sequentially through the flat array of particles and performs a strictly constant $O(1)$ amount of mathematical operations per point, the overall time complexity is $O(N)$, where N is the number of scattered points. While the grid dimensions ($M \times M$) dictate the space complexity, they have negligible impact on execution time since array indices are calculated instantaneously via direct mapping rather than searching.

2. Estimate of Arithmetic Operations per Particle

Based on our highly optimized `utils.cpp` source code, we can calculate the exact arithmetic overhead required to process a single particle:

- **Grid Indexing:** Calculating `i` and `j` indices takes 2 multiplications (`x * inv_dx`).
- **Boundary Clamping:** Accomplished via ternary logical checks, requiring 0 arithmetic operations.
- **Relative Distances:** Calculating `d_x` and `d_y` takes 2 multiplications and 2 subtractions. Inverting them takes 2 subtractions.
- **Fractional Weights:** Calculating the 4 corner weights requires 8 multiplications.
- **Array Addressing:** Finding the 1D base index takes 1 multiplication and 1 addition.
- **Grid Distribution:** Accumulating the weights to the grid takes 4 additions.

Total: The CPU performs exactly **22 arithmetic operations** per particle. Setup variables like `inv_dx` and `cell_area` are computed once globally, costing 0 ops inside the loop.

3. Memory Access Analysis and Cache Behavior

Our implementation utilizes a linear memory access pattern, which perfectly complements the CPU's cache architecture.

- **The Particle Array:** By keeping the 20M points in a contiguous flat array and traversing it via a sequential `for` loop, we trigger the CPU's **Hardware Prefetcher**. The CPU recognizes the linear read pattern and automatically streams the massive 300+ MB dataset from main RAM into the L1 cache before the loop requests it, masking the memory latency.
- **The Grid Array:** The structured grid (even at its largest 1000×400 size) consumes roughly 3.2 MB of memory. This fits entirely inside the L3 cache of both the Lab PC (18 MB) and the HPC Cluster (15 MB). Consequently, the random-access writes to the 4 bounding corners result in fast cache hits rather than expensive RAM fetches.

4. Further Optimizations (Better Hardware Assumptions)

If permitted to execute beyond serial limits, the architecture scales naturally to parallel paradigms:

- **Shared-Memory Multiprocessing (OpenMP):** We could divide the particle array into chunks across multiple CPU cores. However, because multiple particles may map to the same grid cell simultaneously, we must implement **atomic addition operations** or utilize thread-local grid copies (reduction) to prevent memory race conditions.
- **GPU Offloading (CUDA):** The lack of loop dependencies makes this an "Embarrassingly Parallel" problem. We could load the particle array into GPU VRAM and assign each particle to an independent CUDA thread, executing millions of interpolations concurrently.

5. Lab PC vs. HPC Cluster Performance Differences

As observed in the grouped bar chart, execution time scales linearly with particle count (e.g., Configuration D takes significantly longer than C). However, the **Lab PC executed the interpolation significantly faster than the HPC cluster across all configurations**.

- **The Reason:** Our code is strictly serial, meaning it relies entirely on the processing power of a *single core*. The Lab PC's Intel i5 processor is designed with a high base/boost clock speed (up to 4.6 GHz) to handle single-threaded tasks rapidly. Conversely, the HPC's Xeon processors are designed for massive parallel throughput, distributing power across 24 threads at a much lower individual clock speed (2.4 GHz). A single Xeon core simply cannot outpace a modern desktop core in a serial footrace.

6. Pictorial Implementation Approach

The following diagrams illustrate how the underlying data is structured, mapped, and mathematically distributed.

Logical 2D Grid ($N_x = 3, N_y = 2$)

Cell 3	Cell 4	Cell 5
Cell 0	Cell 1	Cell 2

Flattened →

1D mesh_value Array in RAM

Idx 0	Idx 1	Idx 2	Idx 3	Idx 4	Idx 5
Val 0	Val 1	Val 2	Val 3	Val 4	Val 5

Index Formula: $\text{idx} = j * \text{GRID_X} + i$

Figure 4: Data Layout Strategy: Mapping the 2D spatial grid into a contiguous 1D memory array ensures $O(1)$ direct access and cache efficiency.

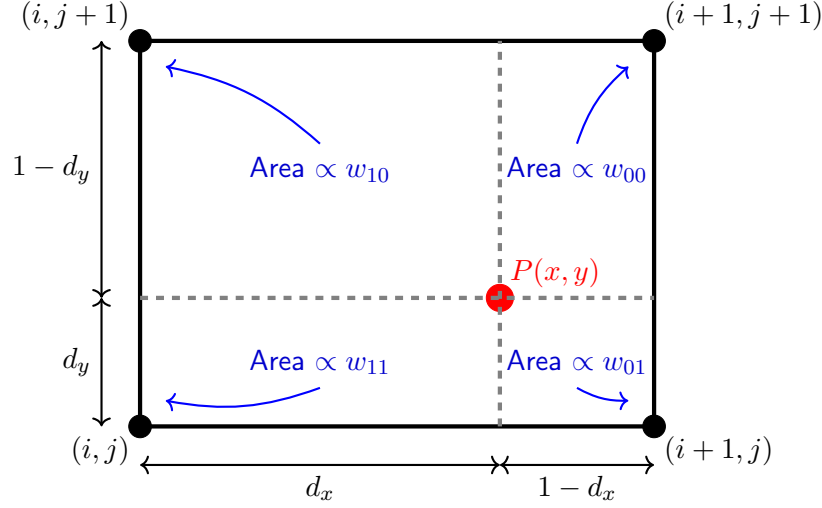


Figure 5: Bilinear Interpolation Mechanism: The scattered point P divides the grid cell into four rectangles. The value distributed to a specific corner is directly proportional to the area of the rectangle *diagonally opposite* to it.

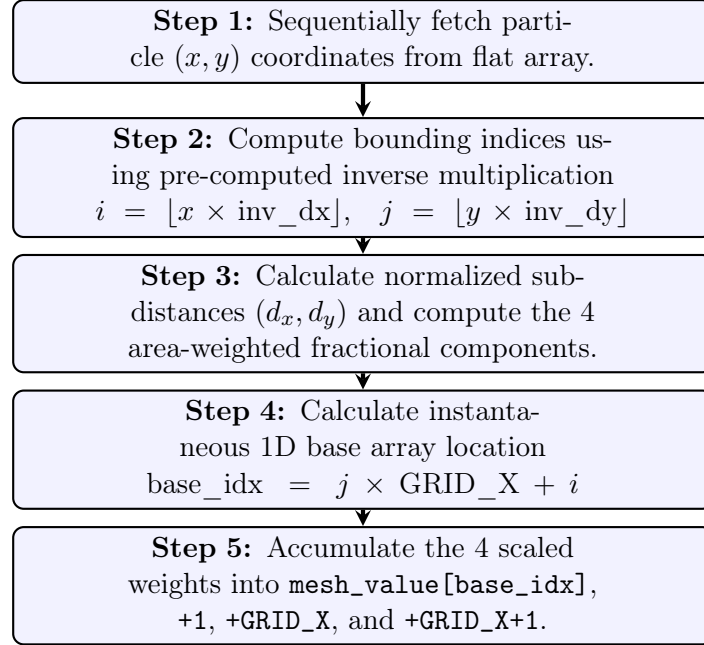


Figure 6: Algorithmic Workflow: The step-by-step mapping procedure executed sequentially for every scattered particle in the domain.

7 Conclusion

The results of this lab demonstrate that algorithmic complexity and memory hierarchy are deeply intertwined. While grouping particles via linked lists logically attempts to improve spatial locality, maintaining a basic flat array architecture dramatically outperformed it. The contiguous memory design eliminates branch-heavy pointer chasing, allowing the CPU's hardware prefetcher to continuously saturate the L1 cache with data. Furthermore, optimizing the core math by substituting division for inverse multiplication and utilizing ternary clamping heavily reduced processing cycles. Ultimately, the performance gap between the Lab PC and the HPC cluster proves that for strictly serial, non-vectorized execution, maximizing a single core's clock speed and cache availability is vastly superior to raw core-count capacity.